

OBSERVABILITY

Logs, Metrics, Traces & Visualization

Building an observability stack with Loki, Prometheus and Grafana on Kubernetes

LOKI · Logs

PROMETHEUS · Metrics

TRACES

GRAFANA · Visualization

What is Observability?

Observability is the ability to understand what is happening inside a system by examining the data it produces — without having to ship new code to ask a new question.



Logs

Discrete, timestamped records of events. “What happened, and what did it say?”



Metrics

Numeric measurements over time. “How much, how fast, how often?”



Traces

The path a single request takes across services. “Where did the time go?”



Visualization

Turns raw signals into dashboards a human can act on.

The Stack We're Deploying



Loki + Promtail

Loki stores and indexes logs. Promtail runs on every node, tails container logs, and ships them to Loki.



Prometheus + Exporters

Prometheus scrapes metrics from apps and infrastructure and stores them as time series.



Grafana

Connects to both Loki and Prometheus as data sources and renders everything in dashboards.

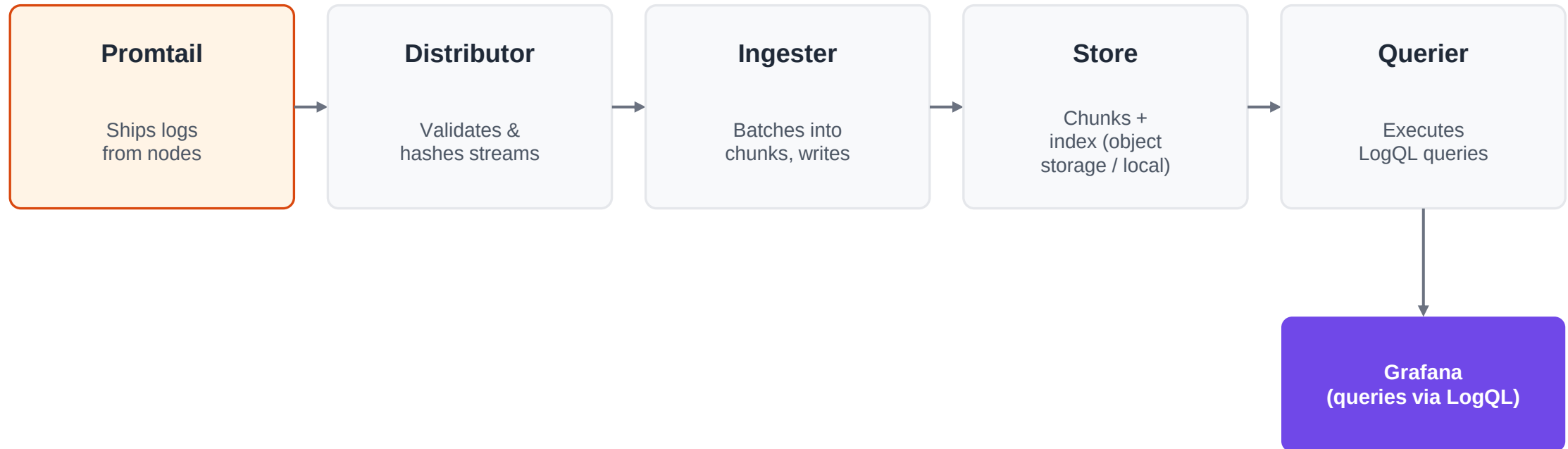
Loki: Log Aggregation, Prometheus-Style

- A horizontally-scalable, multi-tenant log aggregation system built by Grafana Labs.
- Unlike Elasticsearch, Loki does NOT index the full text of every log line — only metadata (labels), which makes it far cheaper to run.
- Logs are compressed into chunks and stored in cheap object storage (or a local filesystem in our lab).
- You query it with LogQL, a query language deliberately similar to PromQL.

Why Loki instead of ELK?

- ✓ 10–20x cheaper storage — index labels, not text
- ✓ Same label model as Kubernetes & Prometheus
- ✓ Native Grafana integration — no separate UI
- ✓ Simple to run for small/medium clusters

Loki Architecture



In our cluster: Promtail runs as a DaemonSet (one pod per node) and reads container log files directly from each node's filesystem.

Promtail: How Logs Reach Loki

1

Discover

Runs as a DaemonSet — one pod per Kubernetes node — and uses Kubernetes service discovery to find every pod on that node.

2

Tail

Reads container log files from `/var/log/pods` on the node's filesystem, following them like ``tail -f``.

3

Label

Attaches Kubernetes metadata as labels: namespace, pod, container, node — the same labels Prometheus uses.

4

Ship

Batches log lines and pushes them to Loki's distributor over HTTP.

Prometheus: Metrics & Time-Series

- An open-source monitoring system that pulls (scrapes) metrics from targets on a regular interval — it does not wait for apps to push data.
- Stores every measurement as a time series: a metric name + key/value labels + a sequence of timestamped values.
- Comes with PromQL, a powerful query language for aggregating, filtering, and computing rates over time series.
- Pairs with Alertmanager to turn PromQL conditions into routed alerts (Slack, email, PagerDuty, etc.).

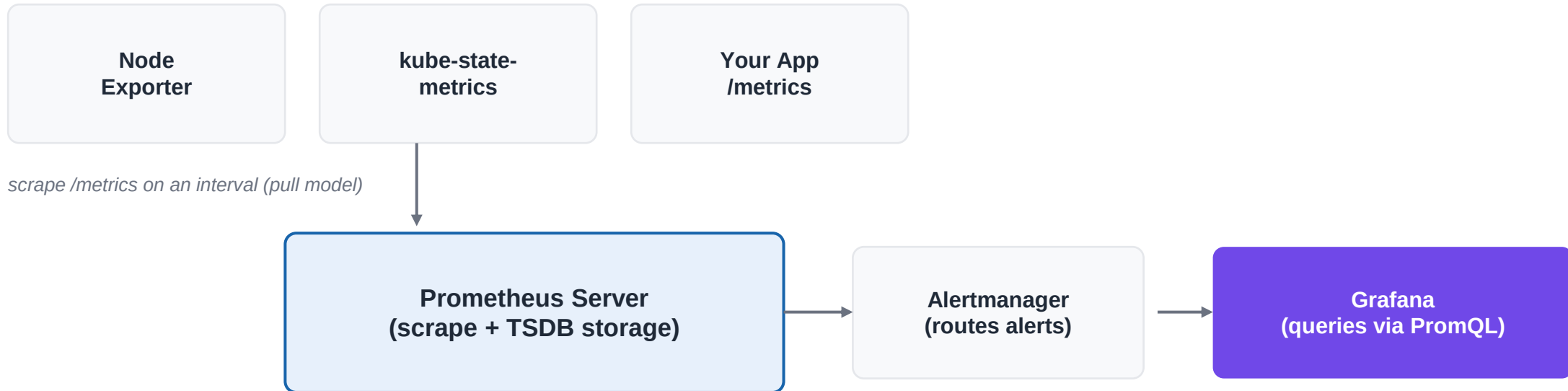
Example time series

```
http_requests_total{
  method="GET",
  status="200",
  pod="new-app-nti-7d9"
} => 1834
```

PromQL example:

```
rate(http_requests_total[5m])
```

Prometheus Architecture



In our cluster: Prometheus finds targets automatically via Kubernetes service discovery and ServiceMonitor/PodMonitor custom resources — no manual IP lists.

Where Kubernetes Metrics Come From



node-exporter

DaemonSet — one pod per node. Exposes host-level metrics: CPU, memory, disk, network.



kube-state-metrics

Watches the Kubernetes API and exposes object state as metrics: pod status, deployment replicas, restarts.



cAdvisor (built into kubelet)

Exposes per-container resource usage (CPU/memory actually consumed by each container).

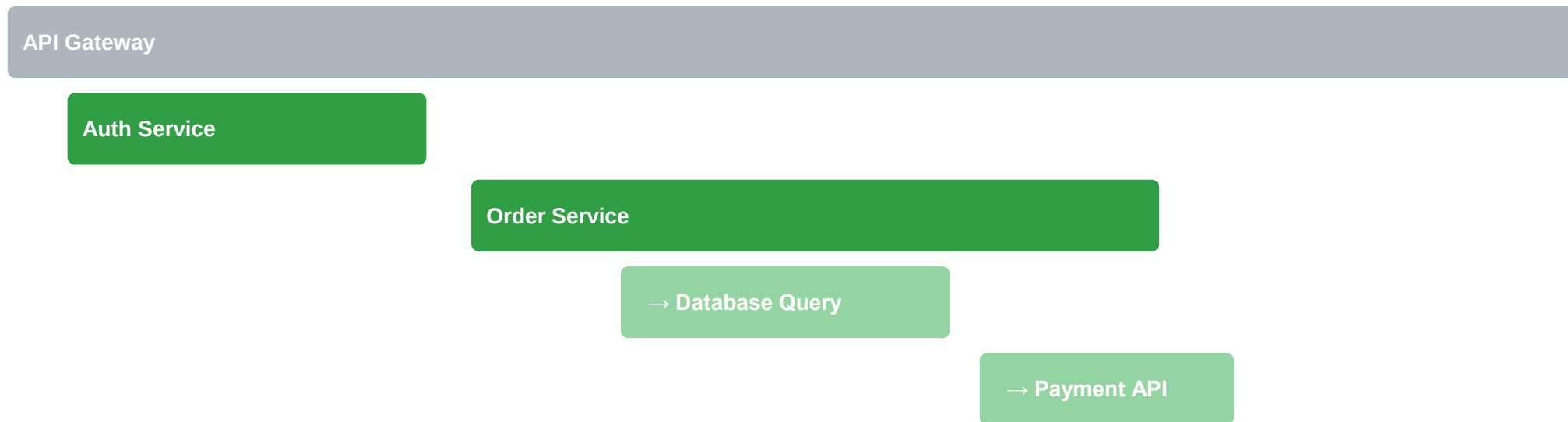


ServiceMonitor / PodMonitor

Custom Resources (CRDs) that tell Prometheus which Services/Pods to scrape — this is how the Helm chart auto-wires everything.

Traces: Following a Request

A trace records the full journey of one request as it moves through multiple services — showing exactly where time was spent.



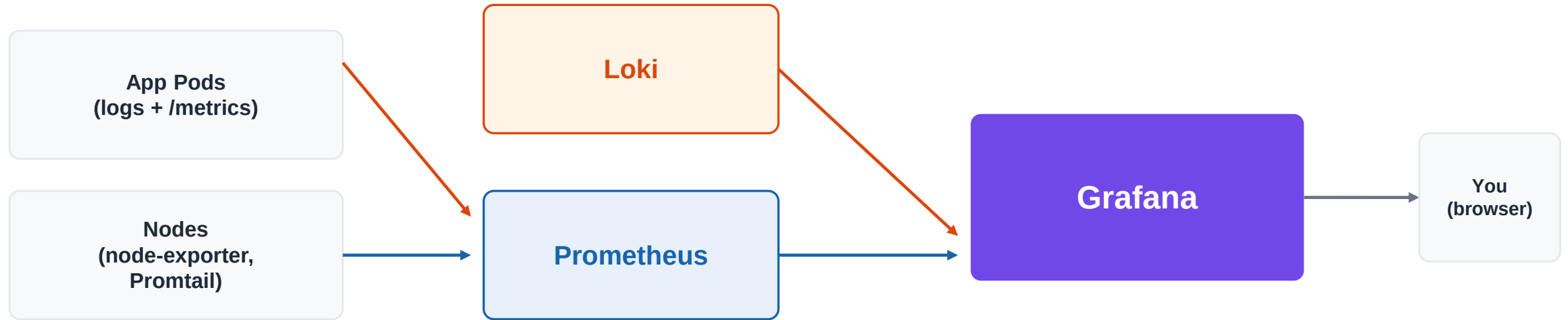
Not deployed in this lab

Tracing normally uses Grafana Tempo, Jaeger, or Zipkin, with apps instrumented via OpenTelemetry. We're focusing on logs + metrics + visualization first — tracing is a natural next step.

Grafana: One Pane of Glass

- Grafana doesn't store data itself — it connects to data sources (Prometheus, Loki, and many others) and queries them live.
- A Data Source is a configured connection: URL + auth + the query language it should speak (PromQL for Prometheus, LogQL for Loki).
- A Dashboard is a collection of Panels; each panel runs a query against a data source and renders it (graph, table, log stream, gauge...).
- Explore mode lets you run ad-hoc PromQL/LogQL queries without building a dashboard — great for debugging.
- Dashboards can be written as JSON and version-controlled, or imported from the community library (grafana.com/dashboards).

How Everything Fits Together



All four components run inside the minikube cluster; only Grafana's port is forwarded out so you can view it from your browser.

Deploying with Helm Charts

kube-prometheus-stack

One Helm chart that installs Prometheus, Alertmanager, Grafana, node-exporter, kube-state-metrics — plus a set of default dashboards.

loki-stack

One Helm chart that installs Loki and Promtail together, and can auto-register Loki as a Grafana data source.

Helm

A package manager for Kubernetes — a “chart” bundles all the manifests (Deployments, Services, ConfigMaps, CRDs) a piece of software needs.

Full command-by-command deployment steps are in the companion guide ([observability-deployment.md](#)).

Opening Grafana in the Browser

- ▶ Helm installs Grafana as a ClusterIP Service inside minikube — not reachable directly from Windows/WSL.
- ▶ Run `kubectl port-forward` to tunnel Grafana's port 80 to your machine's localhost:3000.
- ▶ Open `http://localhost:3000` in your browser and log in (default admin credentials from the chart).
- ▶ Both Prometheus and Loki are already wired up as data sources — open the pre-installed dashboards under Dashboards → Browse.

```
kubectl port-forward --address 0.0.0.0 svc/kube-prometheus-stack-grafana 3000:80
```

Recap

Logs → Loki + Promtail

Metrics → Prometheus + Exporters

Traces → (next lab) Tempo/Jaeger

Visualization → Grafana

Questions?